Medium
Search
Write
Home
Library
Profile
Stories
Stats
Following
Data Science Colo...
Recipes
AI Papers Academy
eBooks
TDS Archive
Lucas de Lima Nogue...
Maximum Genius
Find writers and publications to follow
See recommendations
Get unlimited access to the best of Medium for less than $1/week. Become a member

Detecção Facial com MTCNN

Lucas Kido · Follow · 5 min read · Oct 4, 2020

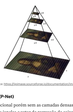
1 · 1

O que é MTCNN?

MTCNN (Multi-task Cascaded Convolutional Networks) é um framework proposto no artigo "Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks" em que tem como base a correlação dos métodos de detecção facial e alinhamento facial.

Framework

Inicialmente a imagem é redimensionada em diferentes escalas para formar uma pirâmide, igual a imagem ao lado, que servirá de entrada para os três estágios da rede.



source: https://miro.medium.com/max/1400/0*ZMPMHZ7KYqqqqqJ

Proposal Network (P-Net)

A uma rede convolucional postém sem as camadas densas e tem o propósito de obter as posições janelas e vetor de regressão de caixa delimitadores, que são as coordenadas do vértice do quadrado onde enquadra a face detectada.

Após isso é feito o método *non-maximum suppression (NMS)* para juntar as janelas que são altamente sobrepostas e com isso a saída será utilizada para alimentar o segundo estágio.

Refine Network (R-Net)

Todas as posições janelas escolhidas pela P-Net servem de entrada para essa rede, que agora possui camadas densas após as convoluções como demonstram a imagem abaixo.

Essa etapa é responsável por diminuir o número de falsas posições janelas e calibrar o vetor de regressão de caixa delimitadoras e também NMS como no estágio passado.

A R-Net tem como saída de a imagem tem 4 uma face ou não, a coordenadas das vértices do box e 10 pontos de referência da face.

Output Network (O-Net)

Similar a R-Net, porém foca em descrever a face em mais detalhes, em particular a rede irá devolver 5 posições das referências faciais.

Treinamento

Como temos na saída 3 elementos (classificação do rosto, coordenadas do box e as coordenadas das referências faciais), para o treinamento temos 5 pontos que é explicado no artigo.

Classificação facial

É um problema de classificação binária, então foi usado *cross-entropy loss*

$$L_i^{det} = -(y_i^{det} \log(p_i) + (1 - y_i^{det})(1 - \log(p_i)))$$

Equação da cross-entropy loss

Onde p_i é a probabilidade que a rede diz que é um rosto ou não, e $y_i det$ é o rótulo com a resposta.

Regressão da caixa delimitadora

Neste caso como é uma regressão foi utilizado a distância euclidiana, que é a diferença do valor predito e o valor real.

$$L_i^{box} = \|\hat{y}_i^{box} - y_i^{box}\|_2^2$$

Equação da distância euclidiana

Localização de pontos faciais

Assim como a regressão da caixa delimitadora, a métrica utilizada é a distância euclidiana, porém são 10 valores ao invés de 4.

$$L_i^{landmark} = \|\hat{y}_i^{landmark} - y_i^{landmark}\|_2^2$$

Equação da distância euclidiana

Treinamento multi-fonte

Como foi utilizado diferentes métricas e treinamentos para diferentes tarefas em cada CNN foi utilizado a seguinte fórmula para ser minimizada.

$$\min \sum_{i=1}^{N} \sum_{j \in \{det, box, landmark\}} \alpha_j \beta_i^j L_i^j$$

Onde N é o número de amostras, α é um vetor composto da importância de cada tarefa e β é indicador da amostra.

Mineração online de amostras difíceis

Diferente do método tradicional onde depois que o classificador é treinado é feito um treinamento apenas com as amostras mais difíceis, na MTCNN em cada *mini-batch* é ordenado a perda na fase de *forward propagation* e escolhido as 70% com a perda mais alta. Então só é computado o gradiente apenas de 70% na fase de backward propagation, ou seja, as amostras mais fáceis são ignoradas para fornecer a modelo.

Utilizando a biblioteca

Passado toda parte teórica chegou a parte mais interessante, a parte prática. Primeiro vamos importar as bibliotecas e a imagem de base.

```
import numpy as np
import cv2
from mtcnn.mtcnn import MTCNN
from matplotlib.patches import Rectangle
from matplotlib.patches import Rectangle, Circle

img = cv2.cvtColor(cv2.imread("henry_cavill.jpg"), cv2.COLOR_BGR2RGB)
plt.imshow(img)
```

Com tudo feito e a imagem do Henry Cavill carregada, vamos carregar o modelo e colocar as saídas na variável result.

```
detector = MTCNN()
result = detector.detect_faces(img)
```

Agora conseguimos ver qual a saída do modelo.

```
[ {
    'box': [436, 206, 423, 560],
    'confidence': 0.9996691022542608,
    'keypoints':
        {
        'left_eye': (552, 426),
        'right_eye': (752, 431),
        'nose': (652, 544),
        'mouth_left': (556, 622),
        'mouth_right': (746, 626)
        }
    }
]
```

Com o campo box temos as coordenadas x1, y1 e a largura e comprimento do retângulo. Já em *confidence* temos a probabilidade que é uma face e em *keypoints* temos as coordenadas do principais pontos da face (olho esquerdo e direito, lado esquerdo e direito da boca e o nariz).

Depois da resposta da rede conseguimos anotar na imagem para destacar os pontos de interesse, que no caso é a face.

```
x1, y1, width, height = result[0]['box']
x2, y2 = x1+width, y1+height
ax = plt.gca(),cmap=cm.jet()
plt.imshow(img)
ax.add_patch(Rectangle((x1, y1), width, height, fill=False, ec='r',
plt.axis('off')
plt.savefig('henry_bbox.png', pad_inches=0.1)
plt.show()
```

Só falta inserirmos os pontos de interesse.

```
left_eye, right_eye = result[0]['keypoints']['left_eye'], result[0]
('keypoints', 'nose')
mouth_left, mouth_right = result[0]['keypoints']['mouth_left'],
result[0]['keypoints']['mouth_right']
x, y = [left_eye[0], right_eye[0], nose[0]
fig = plt.figure()
ax = fig.add_subplot(111)
ax.imshow(img)
ax.add_patch(Rectangle((x1, y1), width, height, fill=False, ec='r',
ax.scatter(x, y)
ax.add_patch(Circle(left_eye, fill=True, color='red', radius=10))
ax.add_patch(Circle(right_eye, fill=True, color='red', radius=10))
ax.add_patch(Circle(nose, fill=True, color='red', radius=10))
ax.add_patch(Circle(mouth_left, fill=True, color='red', radius=10))
ax.add_patch(Circle(mouth_right, fill=True, color='red', radius=10))
ax.axis('off')
plt.savefig('henry_landmark.png', pad_inches=0.1)
plt.show()
```

Tudo feito e se quiser ver o código está no meu github.

Referências

Zhang, K., Zhang, Z., Li, Z., and Qiao, Y. (2016). Joint face detection and alignment using multitask cascaded convolutional networks. IEEE Signal Processing Letters, 23(10):1499–1503

https://arxiv.org/pdf/1604.02878v1

Machine Learning · Facial Detection · Mtcnn

1

Written by Lucas Kido
0 Followers · 0 Following
Follow

No responses yet
Customize banners
What are your thoughts?
More from Lucas Kido
Lucas Kido
Detectando transações fraudulentas em serviços de...
Introdução
Nov 9, 2020
Lucas Kido
Recomendador de Apartamentos
Recommendation system based on Cosine Similarity
Sep 14, 2020 · 1
Lucas Kido
Verificação Facial com ResNet
Introdução
Oct 19, 2020
See all from Lucas Kido
Recommended from Medium
Stanford Just Killed Prompt Engineering With 8 Words (And I...
Stanford just killed prompt engineering with 8 words. No more "You are an expert in..." No more "Let's think step by step." No more...
Oct 10, 2025 · 2K · 102
Google Is Quietly Dismantling Everything OpenAI Built
The most dangerous failure in Silicon Valley isn't bankruptcy. It's becoming the stepping...
Feb 17 · 3.6K · 147
As a Neuroscientist, I Quit These 3 Morning Habits That Destroy You...
Most people do 47 either 10 minutes of meditation or 20 minutes of exercise. Neuroscientist Andrew Huberman says...
Jan 19 · 3.6K · 100
Why Thousands Are Buying Mac Minis to Escape Issues with Big...
Something strange happened in early 2026. Apple stores started running low on Mac...
Feb 15 · 4.4K · 71
Why Are Smart People in Tech Are Quietly Panicking Right Now
The video is blowing up, and it's the smartest version of ChatGPT is hiding the terrifying...
Feb 25 · 1.2K · 100
The AI Bubble Is About To Burst, But The Next Bubble Is Already...
Nothing was wrong with the dot-com bubble. Companies were burning cash, stock prices were...
Mar 16, 2026 · 1.6K · 100
See more recommendations
Help · Status · About · Careers · Press · Blog · Privacy · Rules · Terms · Text to speech